

Building the Full App — Skills & Multi-Agent Orchestration

Two things in one place: 1. **The skills (competency domains)** needed to finish everything in `ROADMAP.md`. 2. **Copy-paste prompts** that make Claude deploy sub-agents to **code → review → test → finalize**, with the project's guardrails baked in.

Part 1 — Skills needed to finish the full app

Grouped by domain, mapped to the roadmap parts. "In repo today" = a foundation already exists to build on.

#	Skill domain	Used for (roadmap)	In repo today
1	Python backend (FastAPI, SQLAlchemy 2.0, Pydantic, async)	All API/services work; Part 1 groundwork; agent features	yes yes
2	PostgreSQL + pgvector (hybrid search, query tuning, Alembic migrations)	Search quality, reranking, migrations (§2.3)	yes yes (migrations: partial)
3	RAG / retrieval engineering (chunking, embeddings, reranking, eval harness)	Retrieval quality, confidence calibration, reranker (§2.5)	yes yes
4	LLM application / prompt engineering (grounded generation, structured output, intent, clustering)	Answer card, Briefing, draft generator, sentiment, strategy advisor	yes yes
5	Speech/vision ML (Whisper transcription, OCR, GPU workers)	Ingestion enrichment at scale	yes adapters; scale partial
6	Frontend (Qwik/Qwik City, TypeScript, SSR, mobile/in-app-browser, design)	Fan page, creator dashboard, approval inbox, Radar UI	yes fan page; dashboard —
7	OAuth2 / identity (Instagram & YouTube OAuth, token refresh, sessions, RBAC)	Onboarding & auth (§2.1)	helpers only partial
8	Security (token encryption/secrets, authz, input validation, abuse/rate limiting)	§2.1, §2.3, §2.4	partial partial
9	Payments / billing (Stripe subscriptions, webhooks, metering, dunning)	Billing & plans (§2.2)	—
10	DevOps / infra (Docker, cloud deploy, managed Postgres/object storage, TLS, CDN)	Production deploy (§2.3)	compose only partial
11	CI/CD & observability (pipelines, Sentry/logging, uptime, cost dashboards, backups)	§2.3	—
12	Platform API integration (Meta Graph/Instagram Messaging, webhooks, YouTube Data API, pagination/rate budgets)	Ingestion, comment-to-DM, App Review	yes yes
13	Job queues & scheduling (RQ/Celery, durable scheduler, retries, dead-letter)	Reliability (§2.3); agentic DM	yes yes; hardening partial
14	Email / messaging delivery (SES/Postmark/Resend)	The Briefing / weekly digest	stub partial
15	Product analytics / data (event pipeline, PostHog/warehouse, dashboards)	Metrics, optimisation, forecasting	events table partial
16	Legal / compliance (GDPR deletion/export, privacy policy, Meta App Review, DPAs)	§2.4	partial partial
17	Testing / QA (pytest, frontend E2E, load/perf testing)	Quality across all parts	backend yes ; FE/load —
18	Product / UX design (dashboard IA, approval flows, briefing layout)	Creator-facing surfaces	—
19	Rust / Axum (later, hot paths only)	Webhook & search-serving at scale (Part 7)	— (deferred by design)
20	Third-party SaaS integration (Notion/Asana/Linear, CRM tools)	Operations & ecosystem (§3.5, v3)	—

Claude Code skills/tools I'll lean on during the build: `code-review` and `security-review` (review stage), `verify` and `run` (test/observe the app), the `eval/` harness (`cci-eval ... --gate`), and `pytest` + `ruff` (CI gate).

Honest read: ~80% of the remaining work needs skills **1-8, 12-13, 17** — all of which already have a live foundation in the repo, so it's extension, not greenfield. The genuinely new external-skill areas are **billing (9)**, **production infra/observability (10-11)**, and **legal (16)**.

Part 2 — The orchestration prompt

Paste this, filling in the one blank (`<<WHAT TO BUILD>>`). It tells Claude to run a code → review → test → finalize pipeline with sub-agents and the project guardrails. Starting it with the word `ultracode` (kept below) is the explicit opt-in that lets Claude use the deterministic multi-agent workflow engine; remove it and Claude will use ad-hoc sub-agents instead.

Master prompt (template)

ultracode

Build the following for Sift using a multi-agent workflow, on branch `claude/creator-content-search-plan-h6o6kf` in an isolated git worktree:

`<<WHAT TO BUILD>>`

(e.g. "Part 1 groundwork: demand-item state machine + outcome-linked event log + voice-memory capture" – or a specific `$/feature` from `docs/ROADMAP.md`)

First read `docs/ROADMAP.md`, `docs/IMPLEMENTATION_PLAN.md`, `README.md`, and the relevant existing code under `apps/` and `packages/`. Match existing conventions exactly: creator-scoped models, provider interfaces for every AI call, approval-mode invariants, deletion-first indexing, the events log.

Run these stages, fanning out sub-agents and scaling the fan-out to the size of the work:

1. **PLAN** – one architect sub-agent reads the relevant code and returns a task breakdown: files to add/change, schema/migration changes, API endpoints, UI, tests, and explicit acceptance criteria tied to the roadmap item.
2. **CODE** (parallel) – one coding sub-agent per independent component from the plan, each in worktree isolation so they don't conflict. Real, complete implementations matching surrounding style and docstrings – no TODO stubs. Each returns the files it changed and why.
3. **REVIEW** (parallel, adversarial) – for each component, a reviewer that tries to find REAL bugs (not rubber-stamp): correctness, security (authz, injection, secret/token handling, creator-scoping leakage), and the compliance invariants below. Also flag reuse/simplification. Fix findings before proceeding. Use the code-review and security-review skills.
4. **TEST** – test sub-agents **WRITE** then **RUN** tests: `pytest` against real `Postgres+pgvector`, plus frontend tests for any UI. Gate to pass before finalize: all tests green, ``ruff check .`` clean, and the eval harness citation/no-answer gate passes (``cci-eval run <handle> --gate``). No comment-to-DM automation path ships unless that gate passes.
5. **FINALIZE** – one integrator merges the worktrees, re-runs the full suite + lint + eval, updates the relevant docs/status in `docs/ROADMAP.md`, and commits with a descriptive message. Open a PR ONLY if I explicitly ask. Report what shipped, what's deferred, and any gate that failed.

Invariants every sub-agent MUST respect:

- Grounded-or-silent: answers cite sources; low confidence → no-answer / archive-link fallback; never a fabricated or uncited answer.
- Approval mode by default for comment-to-DM; never auto-send.
- Compliance: one private reply per comment within 7 days; honor the hourly DM cap; official APIs only; never scrape.
- Every row creator-scoped; audience data pseudonymous; deleting a post purges its chunks from the index.
- Every AI call (LLM/embeddings/transcription/OCR) goes through the provider interfaces – no vendor SDK imported directly.
- Stay on the feature branch; do not push elsewhere; no PR unless asked.

Stop and ask me before proceeding if a decision is ambiguous or architecturally significant. Report token/agent usage at the end.

Ready-made instantiations

A. The immediate next step (Part 1 groundwork):

Replace `<<WHAT TO BUILD>>` with: "Part 1 v1 groundwork from docs/ROADMAP.md: (1) a demand-item state machine on DemandTopic (idea → drafting → published → loop_closed) with transition API + events; (2) extend the events log into outcome-linked records tracing query/comment → answer served → creator action → measured result; (3) a voice-memory table capturing approved replies/hooks and draft edits. Additive only — do not change v1 behaviour."

B. Make-it-real (Phase A, production P0s):

"Phase A production P0s from docs/ROADMAP.md §2: Alembic migrations, secrets management, token encryption at rest, IG OAuth sign-in flow + token refresh job, and the Radar review UI + DM approval inbox. Wire real provider config behind env vars. Deployment manifests for a cloud target."

C. A single v1.5 feature (example):

"The Draft Generator (§3.2) from docs/ROADMAP.md: from a Demand Radar cluster, generate a grounded reel script + hooks using the creator's own content and the audience's literal phrasing, behind the LLM provider interface, with a creator approval step and tests."

Tips for driving it

- **One roadmap item per run.** Smaller scope = better sub-agent results and easier review. Chain runs across phases rather than one giant prompt.
- **Drop `ultracode`** for small/mechanical items to use lighter ad-hoc sub-agents instead of the full workflow engine.
- **Add `open a PR`** explicitly when you want the change up for review on GitHub (default is commit-to-branch only).
- **The gates are non-negotiable** by design — if the eval or test gate fails, the run reports it instead of shipping, which is the intended behaviour.